School of Computer Science Engineering and Information Systems

Winter Semester 2025-2026

Department of Computer Science

UCSC398J – Project-I

Review -2

Date: 17.03.2025

Leaf-Based Plant Disease Classification Using Machine Learning

23BCS0151 – Harish.S

Under the Guidance of

Dr.Jerart Julus L

Associate Professor, Senior

SCORE

Guide Signature with date

Internal Examiner -1

Signature

Internal Examiner-2

Signature

ABSTRACT:

Agriculture plays a vital role in global food production, and plant diseases significantly affect crop yield and quality. Early detection of plant diseases is essential for minimizing crop losses and improving agricultural productivity. Traditional disease detection methods rely on manual inspection by agricultural experts, which is time-consuming, costly, and not always feasible for large-scale farming.

This project presents a **leaf-based plant disease detection system using machine learning and deep learning techniques**. The system analyzes images of plant leaves to identify diseases based on visual symptoms such as discoloration, texture changes, and structural abnormalities. A dataset containing multiple plant disease classes is used to train the model.

The developed system includes **image preprocessing, feature extraction, and classification using convolutional neural networks (CNN)** to achieve high accuracy. Additionally, a **mobile application dashboard** is developed to allow users to upload leaf images and receive disease predictions instantly. The system provides confidence scores and basic recommendations to help farmers take preventive actions.

The proposed solution aims to support **precision agriculture**, enabling faster disease diagnosis, reducing dependency on manual monitoring, and assisting farmers in maintaining crop health.

Keywords:
Plant Disease Detection, Machine Learning, Deep Learning, CNN, Image Processing, Mobile Application.

1.INTRODUCTION:

Agriculture is a fundamental sector that supports global food security and economic development. However, plant diseases remain one of the major challenges affecting crop productivity. Diseases caused by fungi, bacteria, and viruses can spread rapidly and cause severe damage if not detected early.

Traditional methods of plant disease detection rely on manual inspection by agricultural experts. These methods are often slow, expensive, and impractical for large agricultural fields. Farmers may not always have access to expert advice, which leads to delayed diagnosis and reduced crop yield.

Recent advancements in **machine learning, computer vision, and deep learning** have enabled automated plant disease detection systems using leaf images. Leaves are the most commonly affected parts of plants and provide visible symptoms such as color variations, spots, and texture changes that can be analyzed using image processing techniques.

This project aims to develop a **plant disease detection system that uses machine learning models to classify leaf diseases automatically**. The system also integrates a **mobile application interface**, allowing users to capture or upload leaf images and receive disease predictions instantly.

2.PROBLEM STATEMENT:

Plant disease detection is traditionally performed through manual observation by agricultural experts. This process is time-consuming, requires specialized knowledge, and is difficult to apply to large agricultural fields.

Farmers often struggle to identify diseases early, leading to delayed treatment and significant crop losses. In many rural areas, access to expert guidance is limited, which further complicates disease management.

Therefore, there is a need for an **automated and reliable system that can detect plant diseases quickly using leaf images**. Such a system should be accessible, easy to use, and capable of providing accurate predictions to support farmers in taking timely action.

3.OBJECTIVES:

The main objectives of this project are:

1. To develop a machine learning model for plant disease classification using leaf images.
2. To collect and preprocess a dataset of plant leaf images.
3. To train and evaluate a deep learning model for disease prediction.
4. To build a mobile application interface for real-time disease detection.
5. To provide accurate predictions with confidence scores.

4.SCOPE OF THE PROJECT:

The scope of this project focuses on developing a **leaf-based plant disease classification system using machine learning and deep learning techniques**.

The system is capable of:

- Detecting plant diseases using leaf images
- Classifying multiple plant disease categories
- Providing predictions through a mobile application

Future improvements may include:

- Real-time camera detection
- Integration with IoT agricultural systems
- Explainable AI for disease explanation
- Expansion to more crop species and diseases

5.PROPOSED SYSTEM:

The proposed system consists of several stages:

1 Image Acquisition

Leaf images are collected from publicly available datasets and real-world images.

2 Image Preprocessing

Images are resized, normalized, and enhanced to remove noise and improve quality.

3 Dataset Preparation

The dataset is organized into different disease classes for training and testing.

4 Model Training

A convolutional neural network (CNN) model is trained to learn disease patterns from leaf images.

5 Prediction

The trained model predicts the disease class and confidence score.

6 Mobile Application

A mobile application interface allows users to upload leaf images and receive disease predictions.

6.LITERATURE SURVEY:

S.NO	TITLE	MERITS	DEMERITS
1	Plant Disease Detection Using Deep Learning. D. Tejaswi, T. Sri Vaishnavi, B. Nandini, P. Nuka Raju, and B. Deepika,2024	Uses CNN for accurate plant disease detection and provides a mobile-based solution with realtime results. The system supports multiple crops and achieves high accuracy.	Performance depends on image quality and dataset. Real-world field testing and advanced model comparison are limited.
2	Predicting Diabetes Mellitus With Machine	Provides a large, expert-labeled open-access dataset useful for training machine learning	Images are mostly captured in controlled conditions, which may reduce real-world

	Learning Techniques Quan Zou, Kaiyang Qu, Yamei Luo, Dehui Yin, Ying Ju, and Hua Tang,2018	models for plant disease detection.	generalization.
3	Plant Disease Detection and Classification Techniques: A Comparative Study of the Performances. Wubetu Barud Demilie,2024	Compares various machine learning and deep learning techniques and helps identify effective models for disease detection.	Does not propose a new method and lacks real-time or field-level implementation details.
4	A Systematic Analysis of Machine Learning and Deep Learning Based Approaches for Identifying and Diagnosing Plant Diseases.	Provides a detailed comparative analysis of machine learning and deep learning techniques for plant disease detection using the PlantVillage dataset. Highlights the effectiveness of CNN-based models with high accuracy for multiple crops.	Mainly focuses on analysis and comparison rather than proposing a novel system. Performance is largely dependent on dataset quality and controlled image conditions.
5	Plant Disease Detection Using Machine Learning. Shima Ramesh, Niveditha M., Pooja R., Prasad Bhat N., Shashank N., Ramachandra Hebbar, and P. V. Vinod,2018	Uses Random Forest with HOG and texture features to classify healthy and diseased leaves. The approach is simple, cost-effective, and suitable for small datasets.	Achieves lower accuracy compared to deep learning models and is sensitive to image quality and background conditions.
6	Using Deep Learning for Image-Based Plant Disease Detection. Sharada P. Mohanty, David P. Hughes, and Marcel Salathé,2016	Uses CNN-based models for accurate multi-class plant disease detection with improved performance over traditional methods.	Performance depends on dataset quality and may not generalize well to real field conditions.
7	A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification. Edna Chebet Too, Li Yujian, Sam	Compares multiple deep CNN architectures and shows that DenseNet achieves very high accuracy with fewer parameters.	Uses a controlled dataset and requires high computational resources for training.

	Njukia, and Liu Yingchun,2019		
8	Research on Plant Disease Identification Based on CNN. Xuewei Sun, Guohou Li, Peixin Qu, Xiwang Xie, Xipeng Pan, and Weidong Zhang,2022	Proposes FL-EfficientNet with focal loss and attention mechanism, achieving high accuracy and fast convergence.	Model design is complex and requires careful parameter tuning.
9.	Diabetes Prediction Using Ensembling of Different Machine Learning Classifiers.Md. Kamrul Hasan, Md. Ashraful Alam, Dola Das, Eklas Hossain, and Mahmudul Hasan,2020	Reviews and applies CNN-based models for effective plant disease identification across multiple crops.	Performance depends on controlled datasets and lacks extensive real-world field validation
10	Improving Plant Disease Classification With Deep Learning-Based Prediction Model Using Explainable Artificial Intelligence. Natasha Nigar, Hafiz Muhammad Faisal, Muhammad Umer, Olukayode Oki, and Jose Manappattukunnel Lukose,204	Achieves very high accuracy using EfficientNet and provides transparent predictions through XAI (LIME), improving trust and decision-making.	Requires large datasets and high computational resources; real-time field deployment challenges remain.

6.1.FINDINGS IN LITERATURE SURVEY:

From the literature survey, the following observations are made:

- Deep learning models outperform traditional machine learning methods.

- CNN architectures are widely used for plant disease detection.
- Model performance depends heavily on dataset quality and diversity.
- Many systems lack real-time field validation.
- Mobile-based solutions are increasingly used for farmer accessibility.

These findings support the development of a **deep learning-based plant disease detection system with a mobile interface**.

7.METHEDOLOGY:

The methodology followed in this project includes:

1. Dataset collection and organization
2. Image preprocessing and augmentation
3. Model training using CNN architecture
4. Model evaluation using accuracy and loss metrics
5. Deployment of the trained model into a mobile application
6. Real-time prediction using uploaded images.

7.1 Gap Identification:

Although several studies have applied machine learning and deep learning techniques for plant disease detection, some limitations still remain. Most existing systems rely on **datasets collected in controlled environments**, which may not perform well in real-world field conditions. Many models also focus on **a single crop or a limited number of diseases**, reducing their applicability for farmers who grow multiple crops.

Another limitation is that most systems assume that the input image always contains a plant leaf. In practical situations, users may upload **non-leaf images or images with complex backgrounds**, which existing models fail to handle effectively. In addition, many research works concentrate only on model development and do not provide **mobile-based applications for real-time disease detection**.

To address these gaps, the proposed system develops a **multi-crop plant disease detection model** and integrates it with a **mobile application dashboard** for easy and real-time disease prediction.

8.SOFTWARE REQUIREMENTS:

Functional Requirements

- Upload leaf images
- Detect plant diseases
- Display prediction results
- Show confidence level
- Provide basic disease information

Non-Functional Requirements

- Fast prediction time
- High accuracy
- Easy-to-use interface
- Compatibility with mobile devices

Tools & Technologies

- Python
- TensorFlow / Keras
- OpenCV
- NumPy
- Flutter (Mobile App)
- Google Colab / Local GPU

Functional and Non Functional Requirements

9.SYSTEM ARCHITECTURE:

Fig. 9.1 System Architecture

11. IMPLEMENTATION

11.1. PROJECT PLAN

Phase	Description
Phase 1	Dataset collection
Phase 2	Image preprocessing
Phase 3	Model training
Phase 4	Model evaluation
Phase 5	Mobile app development
Phase 6	Testing and integration

Current Progress:

- Dataset preparation – Completed
- Model training – 80% Completed
- Mobile dashboard – Completed
- Integration – In progress

11.2 SAMPLE CODE

```
import os

# List the contents of the Google Drive root directory
drive_root_path = '/content/drive/MyDrive/'
print(f"Contents of '{drive_root_path}':")
if os.path.exists(drive_root_path):
    for item in os.listdir(drive_root_path):
        print(f"- {item}")
else:
    print(f"The directory '{drive_root_path}' does not exist. Please ensure Google Drive is mounted correctly.")
```

```
Contents of '/content/drive/MyDrive/':
- Colab Notebooks
- Certification
- income_compressed.pdf
- StudentGradeHistory_23BCS0151.pdf
- v4v.pdf
- IMG_20251130_085726_870.jpg
- image processing
- IMG_20260113_124121_114.jpg
- plant_disease_mobilenetv2.h5
- plant_model.keras
- plant_model.tflite
- new dataset
```

```
train_dir = "/content/drive/MyDrive/new dataset/train/Tomato"
val_dir = "/content/drive/MyDrive/new dataset/val/Tomato"
test_dir = "/content/drive/MyDrive/new dataset/test/Tomato"
```

```
import tensorflow as tf

IMG_SIZE = (224,224)
BATCH_SIZE = 16

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 3052 files belonging to 11 classes.
Found 850 files belonging to 11 classes.
Found 856 files belonging to 11 classes.
```

```
class_names = train_ds.class_names
print("Number of classes:", len(class_names))
print("Classes:", class_names)
```

```
Number of classes: 11
Classes: ['Bacterial spot (Tomato)', 'Early blight (Tomato)', 'Late blight (Tomato)', 'Leaf Mold (Tomato)', 'Non_Leaf', 'Septoria leaf spot (Tomato)', 'Spider mites Two-spotted spider mite (Tomato)', 'Target Spot (Tomato)', 'Tomato Yellow Leaf Curl Virus (Tomato)', 'Tomato mosaic virus (Tomato)', 'healthy (Tomato)']
```

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Rescaling(1./255),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),

    layers.Dense(len(class_names),activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20
)
```

```
Epoch 1/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 731ms/step - accuracy: 0.4577 - loss: 1.5567 - val_accuracy: 0.6847 - val_loss: 0.8936
Epoch 2/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 134s 703ms/step - accuracy: 0.7603 - loss: 0.7072 - val_accuracy: 0.7447 - val_loss: 0.7436
Epoch 3/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 132s 693ms/step - accuracy: 0.8406 - loss: 0.4419 - val_accuracy: 0.8141 - val_loss: 0.6380
Epoch 4/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 137s 717ms/step - accuracy: 0.9063 - loss: 0.2782 - val_accuracy: 0.7804 - val_loss: 0.7258
Epoch 5/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 140s 733ms/step - accuracy: 0.9450 - loss: 0.1725 - val_accuracy: 0.8035 - val_loss: 0.8562
Epoch 6/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 140s 735ms/step - accuracy: 0.9620 - loss: 0.1050 - val_accuracy: 0.8235 - val_loss: 0.8753
Epoch 7/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 141s 736ms/step - accuracy: 0.9564 - loss: 0.1530 - val_accuracy: 0.8047 - val_loss: 0.9763
Epoch 8/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 742ms/step - accuracy: 0.9581 - loss: 0.1380 - val_accuracy: 0.7871 - val_loss: 0.9626
Epoch 9/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 743ms/step - accuracy: 0.9803 - loss: 0.0644 - val_accuracy: 0.7388 - val_loss: 1.2714
Epoch 10/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 141s 740ms/step - accuracy: 0.9843 - loss: 0.0467 - val_accuracy: 0.7553 - val_loss: 1.3833
Epoch 11/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 741ms/step - accuracy: 0.9833 - loss: 0.0546 - val_accuracy: 0.7812 - val_loss: 1.1753
Epoch 12/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 741ms/step - accuracy: 0.9843 - loss: 0.0452 - val_accuracy: 0.7529 - val_loss: 1.5772
Epoch 13/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 139s 729ms/step - accuracy: 0.9436 - loss: 0.1789 - val_accuracy: 0.7882 - val_loss: 0.9984
Epoch 14/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 746ms/step - accuracy: 0.9908 - loss: 0.0288 - val_accuracy: 0.7929 - val_loss: 1.2693
Epoch 15/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 143s 749ms/step - accuracy: 0.9839 - loss: 0.0524 - val_accuracy: 0.8012 - val_loss: 1.3182
Epoch 16/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 142s 741ms/step - accuracy: 0.9712 - loss: 0.0957 - val_accuracy: 0.6804 - val_loss: 1.8199
Epoch 17/20
191/191 ━━━━━━━━━━━━━━━━━━━━ 141s 737ms/step - accuracy: 0.9869 - loss: 0.0431 - val_accuracy: 0.7976 - val_loss: 1.3454
Epoch 18/20
```

```
test_loss, test_acc = model.evaluate(test_ds)

print("Test Accuracy:", test_acc)
```

```
54/54 ━━━━━━━━━━━━━━━━━━━━ 14s 247ms/step - accuracy: 0.7336 - loss: 1.8239
Test Accuracy: 0.7336448431015015
```

```
model.save("tomato_model.keras")
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("tomato_model.tflite","wb") as f:
    f.write(tflite_model)
```

```
Saved artifact at '/tmp/tmpsc7gad9z'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor')
Output Type:
  TensorSpec(shape=(None, 11), dtype=tf.float32, name=None)
Captures:
  137546474001536: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474086352: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474106576: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474095440: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474099856: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474099008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474107728: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474108432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474109072: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474095248: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
from google.colab import files
files.download("tomato_model.tflite")
```

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files  No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving non_leafimage3866.jpg to non_leafimage3866.jpg
```

```
import tensorflow as tf

model_path = "/content/drive/MyDrive/models/tomato_model.tflite"

interpreter = tf.lite.Interpreter(model_path=model_path)
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print(input_details)
print(output_details)

import numpy as np
from PIL import Image

img = Image.open("non_leafimage3866.jpg").resize((224,224))

img = np.array(img)

img = img / 255.0

img = np.expand_dims(img, axis=0).astype(np.float32)

interpreter.set_tensor(input_details[0]['index'], img)
interpreter.invoke()

output = interpreter.get_tensor(output_details[0]['index'])

print(output)

import os

dataset_path = "/content/drive/MyDrive/new dataset/train/tomato"

labels = sorted(os.listdir(dataset_path))

print(labels)
```

```
[{'name': 'serving_default_keras_tensor:0', 'index': 0, 'shape': array([  1, 224, 224,   3], dtype=int32), 'shape_signature': array([ -1, 224, 224,   3], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([], dtype=float32), 'zero_points': array([], dtype=int32), 'quantized_dimension': 0, 'block_size': 0), 'sparsity_parameters': {}}]
[{'name': 'StatefulPartitionedCall_1:0', 'index': 26, 'shape': array([ 1, 11], dtype=int32), 'shape_signature': array([-1, 11], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([], dtype=float32), 'zero_points': array([], dtype=int32), 'quantized_dimension': 0, 'block_size': 0), 'sparsity_parameters': {}}]
[[2.2280023e-08 2.7286472e-05 9.9994014e-01 1.8201166e-05 1.4215259e-05
  1.3853594e-08 9.7687760e-14 3.4091814e-13 1.1175260e-11 2.6584547e-11
  2.2112209e-10]]
['Bacterial spot (Tomato)', 'Early blight (Tomato)', 'Late blight (Tomato)', 'Leaf Mold (Tomato)', 'Non_Leaf', 'Septoria leaf spot (Tomato)', 'Spider mites Two-spotted spider mite (Tomato)', 'Target Spot (Tomato)', 'Tomato Yellow Leaf Curl Virus (Tomato)', 'Tomato mosaic virus (Tomato)', 'healthy (Tomato)']
```

```
for i, prob in enumerate(output[0]):
    print(labels[i], ":", prob)
```

```
Bacterial spot (Tomato) : 2.2280023e-08
Early blight (Tomato) : 2.7286472e-05
Late blight (Tomato) : 0.99994014
Leaf Mold (Tomato) : 1.8201166e-05
Non_Leaf : 1.4215259e-05
Septoria leaf spot (Tomato) : 1.3853594e-08
Spider mites Two-spotted spider mite (Tomato) : 9.7687760e-14
Target Spot (Tomato) : 3.4091814e-13
Tomato Yellow Leaf Curl Virus (Tomato) : 1.1175260e-11
Tomato mosaic virus (Tomato) : 2.6584547e-11
healthy (Tomato) : 2.2112209e-10
```

```python
import numpy as np

predicted_index = np.argmax(output)
confidence = output[0][predicted_index]

# Get top 2 predictions
top2 = output[0].argsort()[-2:][::-1]

if output[0][top2[0]] - output[0][top2[1]] < 0.4:
    print("Image unclear. Please capture a clear leaf.")
else:
    print("Prediction:", labels[top2[0]])
    print("Confidence:", output[0][top2[0]])
```

```
Prediction: Late blight (Tomato)
Confidence: 0.99994814
```

```python
train_dir = "/content/drive/MyDrive/new dataset/train/Mango"
val_dir = "/content/drive/MyDrive/new dataset/val/Mango"
test_dir = "/content/drive/MyDrive/new dataset/test/Mango"
```

```python
import tensorflow as tf

IMG_SIZE = (224,224)
BATCH_SIZE = 16

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 2490 files belonging to 9 classes.
Found 731 files belonging to 9 classes.
Found 730 files belonging to 9 classes.
```

```
class_names = train_ds.class_names
print("Number of classes:", len(class_names))
print("Classes:", class_names)
```

```
Number of classes: 9
Classes: ['Anthracnose (Mango)', 'Bacterial Canker (Mango)', 'Cutting Weevil (Mango)', 'Die Back (Mango)', 'Gall Midge (Mango)', 'Healthy (Mango)', 'Non_Leaf', 'Powdery Mildew (Mango)', 'Sooty Mould (Mango)']
```

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Rescaling(1./255),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),

    layers.Dense(len(class_names),activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 171s 1s/step - accuracy: 0.5715 - loss: 1.1984 - val_accuracy: 0.7852 - val_loss: 0.6826
Epoch 2/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 116s 741ms/step - accuracy: 0.8173 - loss: 0.5161 - val_accuracy: 0.8659 - val_loss: 0.4341
Epoch 3/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 115s 737ms/step - accuracy: 0.8827 - loss: 0.3409 - val_accuracy: 0.8741 - val_loss: 0.4438
Epoch 4/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 114s 732ms/step - accuracy: 0.9353 - loss: 0.1957 - val_accuracy: 0.8865 - val_loss: 0.3821
Epoch 5/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 114s 732ms/step - accuracy: 0.9486 - loss: 0.1595 - val_accuracy: 0.8728 - val_loss: 0.4920
Epoch 6/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 114s 732ms/step - accuracy: 0.9482 - loss: 0.1871 - val_accuracy: 0.8851 - val_loss: 0.5555
Epoch 7/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 116s 744ms/step - accuracy: 0.9799 - loss: 0.0743 - val_accuracy: 0.8960 - val_loss: 0.5675
Epoch 8/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 114s 732ms/step - accuracy: 0.9960 - loss: 0.0126 - val_accuracy: 0.8960 - val_loss: 0.5563
Epoch 9/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 115s 735ms/step - accuracy: 0.9823 - loss: 0.0532 - val_accuracy: 0.8947 - val_loss: 0.5943
Epoch 10/10
156/156 ━━━━━━━━━━━━━━━━━━━━ 114s 734ms/step - accuracy: 0.9594 - loss: 0.1389 - val_accuracy: 0.8837 - val_loss: 0.3563
```

```
model.save("Mango_model.keras")
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("Mango_model.tflite","wb") as f:
    f.write(tflite_model)
```

```
Saved artifact at '/tmp/tmpecbdxqpq'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_11')
Output Type:
  TensorSpec(shape=(None, 9), dtype=tf.float32, name=None)
Captures:
  137546474089040: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474082320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474093264: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474088464: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474081360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474093648: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474091344: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474079056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474093072: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474082128: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
train_dir = "/content/drive/MyDrive/new dataset/train/Apple"
val_dir = "/content/drive/MyDrive/new dataset/val/Apple"
test_dir = "/content/drive/MyDrive/new dataset/test/Apple"
```

```
import tensorflow as tf

IMG_SIZE = (224,224)
BATCH_SIZE = 16

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 1393 files belonging to 5 classes.
Found 488 files belonging to 5 classes.
Found 484 files belonging to 5 classes.
```

```
class_names = train_ds.class_names
print("Number of classes:", len(class_names))
print("Classes:", class_names)
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Rescaling(1./255),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),

    layers.Dense(len(class_names),activation='softmax')
])
```

```python
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```python
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 75s 858ms/step - accuracy: 0.7545 - loss: 0.6792 - val_accuracy: 0.7520 - val_loss: 0.6726
Epoch 2/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 76s 862ms/step - accuracy: 0.8600 - loss: 0.4114 - val_accuracy: 0.6475 - val_loss: 1.1736
Epoch 3/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 77s 875ms/step - accuracy: 0.9124 - loss: 0.2596 - val_accuracy: 0.8197 - val_loss: 0.4929
Epoch 4/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 76s 870ms/step - accuracy: 0.9289 - loss: 0.2115 - val_accuracy: 0.8689 - val_loss: 0.3957
Epoch 5/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 71s 803ms/step - accuracy: 0.9526 - loss: 0.1608 - val_accuracy: 0.8238 - val_loss: 0.6598
Epoch 6/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 70s 798ms/step - accuracy: 0.9591 - loss: 0.1298 - val_accuracy: 0.8852 - val_loss: 0.5084
Epoch 7/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 70s 794ms/step - accuracy: 0.9713 - loss: 0.0905 - val_accuracy: 0.8689 - val_loss: 0.5645
Epoch 8/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 64s 733ms/step - accuracy: 0.9648 - loss: 0.1001 - val_accuracy: 0.8074 - val_loss: 0.8590
Epoch 9/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 66s 749ms/step - accuracy: 0.9720 - loss: 0.0878 - val_accuracy: 0.8258 - val_loss: 0.7773
Epoch 10/10
88/88 ━━━━━━━━━━━━━━━━━━━━ 66s 750ms/step - accuracy: 0.9677 - loss: 0.0919 - val_accuracy: 0.7971 - val_loss: 1.0713
```

```python
test_loss, test_acc = model.evaluate(test_ds)

print("Test Accuracy:", test_acc)
```

```
31/31 ━━━━━━━━━━━━━━━━━━━━ 8s 257ms/step - accuracy: 0.6839 - loss: 3.2920
Test Accuracy: 0.68388432264328
```

```python
model.save("Apple_model.keras")
```

```python
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("Apple_model.tflite","wb") as f:
    f.write(tflite_model)
```

```
Saved artifact at '/tmp/tmpsho8l9ud'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_22')
Output Type:
  TensorSpec(shape=(None, 9), dtype=tf.float32, name=None)
Captures:
  137546474084048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474089616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474088656: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474061904: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474064976: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474064400: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474068048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474077840: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474067472: TensorSpec(shape=(), dtype=tf.resource, name=None)
  137546474072848: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```python
from google.colab import files
files.download("Apple_model.tflite")
```

```
train_dir = "/content/drive/MyDrive/new dataset/train/Pepper"
val_dir = "/content/drive/MyDrive/new dataset/val/Pepper"
test_dir = "/content/drive/MyDrive/new dataset/test/Pepper"
```

```
import tensorflow as tf

IMG_SIZE = (224,224)
BATCH_SIZE = 16

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 817 files belonging to 3 classes.
Found 378 files belonging to 3 classes.
Found 379 files belonging to 3 classes.
```

```
class_names = train_ds.class_names
print("Number of classes:", len(class_names))
print("Classes:", class_names)
```

```
Number of classes: 3
Classes: ['Bacterial spot (Pepper, bell)', 'Non_Leaf', 'healthy (Pepper, bell)']
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Rescaling(1./255),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),

    layers.Dense(len(class_names),activation='softmax')
])
```

```python
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```python
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 168s 3s/step - accuracy: 0.5054 - loss: 1.2837 - val_accuracy: 0.8757 - val_loss: 0.2654
Epoch 2/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 114s 2s/step - accuracy: 0.8670 - loss: 0.3076 - val_accuracy: 0.8810 - val_loss: 0.2922
Epoch 3/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 147s 2s/step - accuracy: 0.9244 - loss: 0.1803 - val_accuracy: 0.9418 - val_loss: 0.1341
Epoch 4/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.8872 - loss: 0.3827 - val_accuracy: 0.9286 - val_loss: 0.1835
Epoch 5/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 136s 2s/step - accuracy: 0.9614 - loss: 0.1171 - val_accuracy: 0.9471 - val_loss: 0.2180
Epoch 6/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 147s 2s/step - accuracy: 0.9501 - loss: 0.1013 - val_accuracy: 0.8466 - val_loss: 0.6517
Epoch 7/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 141s 2s/step - accuracy: 0.9654 - loss: 0.1308 - val_accuracy: 0.9339 - val_loss: 0.2291
Epoch 8/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 121s 2s/step - accuracy: 0.9667 - loss: 0.0849 - val_accuracy: 0.9471 - val_loss: 0.1901
Epoch 9/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 141s 2s/step - accuracy: 0.9735 - loss: 0.0627 - val_accuracy: 0.9709 - val_loss: 0.0980
Epoch 10/10
52/52 ━━━━━━━━━━━━━━━━━━━━ 144s 2s/step - accuracy: 0.9986 - loss: 0.0121 - val_accuracy: 0.9497 - val_loss: 0.1977
```

```python
test_loss, test_acc = model.evaluate(test_ds)

print("Test Accuracy:", test_acc)
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 58s 2s/step - accuracy: 0.7331 - loss: 1.3218
Test Accuracy: 0.7229551672935486
```

```python
model.save("Pepper_model.keras")
```

```python
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("Pepper_model.tflite","wb") as f:
    f.write(tflite_model)
```

```
Saved artifact at '/tmp/tmpv_r754w_'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_11')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  132718097918160: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097914128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097917392: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097916432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097916624: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718048074000: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718048073616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718048074384: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097919312: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718048074768: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```python
from google.colab import files
files.download("Pepper_model.tflite")
```

```python
train_dir = "/content/drive/MyDrive/new dataset/train/Pumpkin"
val_dir = "/content/drive/MyDrive/new dataset/val/Pumpkin"
test_dir = "/content/drive/MyDrive/new dataset/test/Pumpkin"
```

```python
import tensorflow as tf

IMG_SIZE = (224,224)
BATCH_SIZE = 16

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 1650 files belonging to 6 classes.
Found 553 files belonging to 6 classes.
Found 556 files belonging to 6 classes.
```

```python
class_names = train_ds.class_names
print("Number of classes:", len(class_names))
print("Classes:", class_names)
```

```
Number of classes: 6
Classes: ['Bacterial Leaf Spot (Pumpkin)', 'Downy Mildew (Pumpkin)', 'Healthy Leaf (Pumpkin)', 'Mosaic Disease (Pumpkin)', 'Non_Leaf', 'Powdery Mildew (Pumpkin)']
```

```python
from tensorflow.keras import layers, models

model = models.Sequential([

    layers.Rescaling(1./255),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),

    layers.Dense(len(class_names),activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 327s 3s/step - accuracy: 0.3452 - loss: 1.7105 - val_accuracy: 0.7378 - val_loss: 0.6543
Epoch 2/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 231s 2s/step - accuracy: 0.6407 - loss: 0.9230 - val_accuracy: 0.7432 - val_loss: 0.6412
Epoch 3/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 249s 2s/step - accuracy: 0.7084 - loss: 0.6969 - val_accuracy: 0.6962 - val_loss: 0.8162
Epoch 4/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 243s 2s/step - accuracy: 0.7907 - loss: 0.5256 - val_accuracy: 0.8246 - val_loss: 0.4242
Epoch 5/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 281s 2s/step - accuracy: 0.8363 - loss: 0.4126 - val_accuracy: 0.8427 - val_loss: 0.4911
Epoch 6/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 263s 2s/step - accuracy: 0.9020 - loss: 0.2847 - val_accuracy: 0.8336 - val_loss: 0.6006
Epoch 7/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 235s 2s/step - accuracy: 0.9503 - loss: 0.1772 - val_accuracy: 0.8354 - val_loss: 0.5765
Epoch 8/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 259s 2s/step - accuracy: 0.9753 - loss: 0.0727 - val_accuracy: 0.8011 - val_loss: 0.7007
Epoch 9/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 221s 2s/step - accuracy: 0.9722 - loss: 0.0861 - val_accuracy: 0.8174 - val_loss: 0.7723
Epoch 10/10
104/104 ━━━━━━━━━━━━━━━━━━━━ 262s 2s/step - accuracy: 0.9993 - loss: 0.0072 - val_accuracy: 0.8336 - val_loss: 0.8955
```

```
test_loss, test_acc = model.evaluate(test_ds)

print("Test Accuracy:", test_acc)
```

```
35/35 ━━━━━━━━━━━━━━━━━━━━ 88s 3s/step - accuracy: 0.8171 - loss: 1.0803
Test Accuracy: 0.8309352397918701
```

```
model.save("Pumpkin_model.keras")
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("Pumpkin_model.tflite","wb") as f:
    f.write(tflite_model)
```

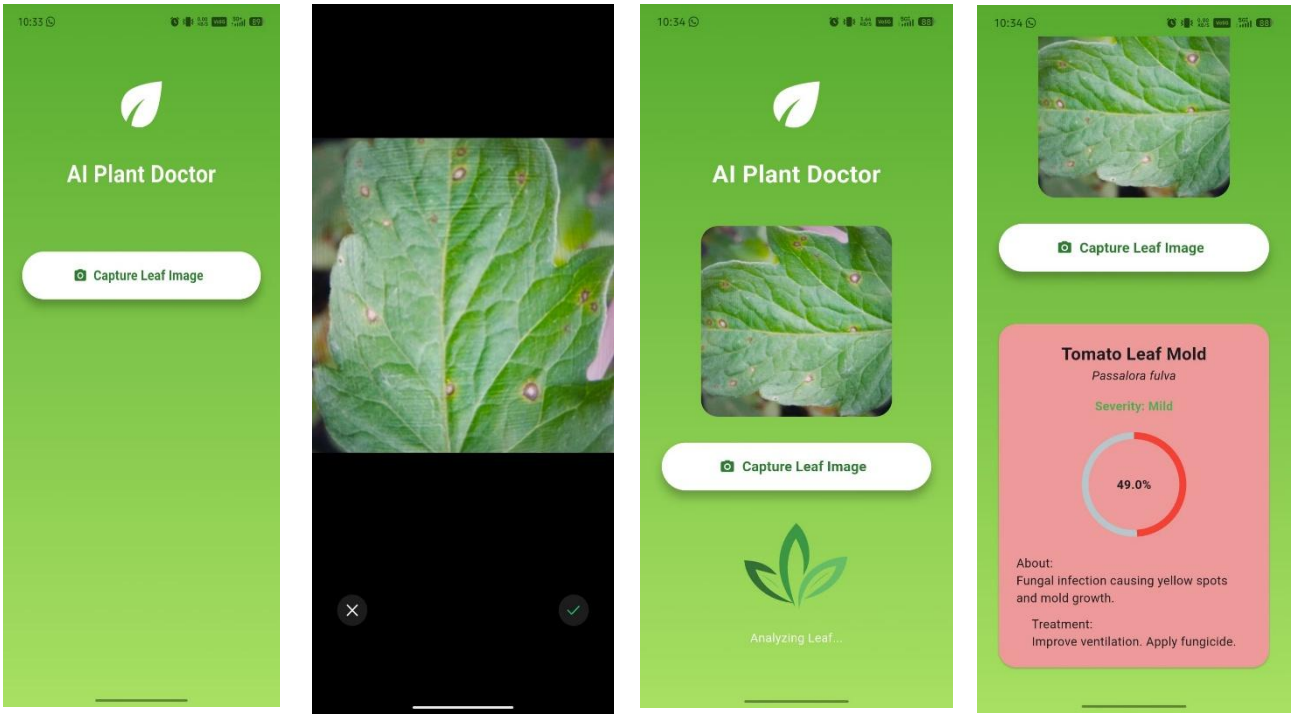
```
Saved artifact at '/tmp/tmp02jbr38w'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_22')
Output Type:
  TensorSpec(shape=(None, 6), dtype=tf.float32, name=None)
Captures:
  132717511919952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511920720: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511921296: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511920336: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511920528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511921872: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511920912: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511922256: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132718097917008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132717511922640: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

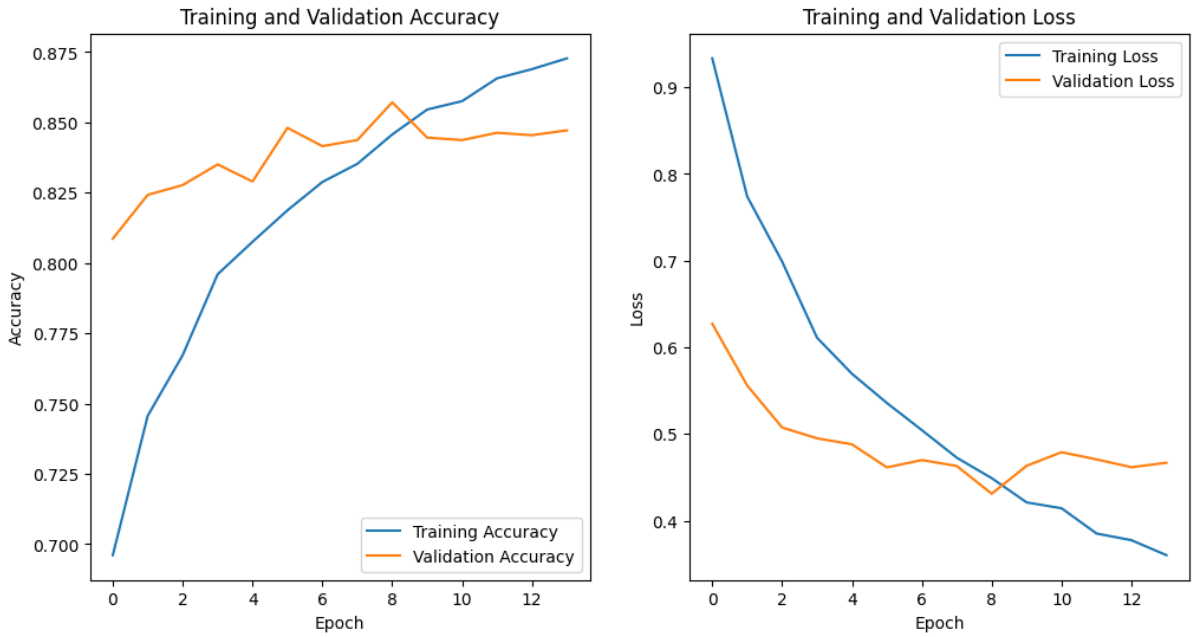
```
from google.colab import files
files.download("Pumpkin_model.tflite")
```

11.3 SAMPLE SCREEN SHOTS

Mobile Application Dashboard



Model training accuracy graph:



12.SUMMARY:

This project presents a **machine learning-based plant disease detection system using leaf images**. The system utilizes deep learning techniques to classify plant diseases accurately. A mobile application interface has been developed to allow users to upload images and obtain predictions in real time.

The model implementation is currently **80% complete**, and the mobile dashboard has been successfully developed. The system demonstrates the potential of artificial intelligence in supporting precision agriculture and helping farmers detect diseases early.

Future work will focus on improving model accuracy, expanding the dataset, and implementing explainable AI techniques.

REFERENCES:

[1] Shima Ramesh, Niveditha M., Pooja R., Prasad Bhat N., Shashank N., Ramachandra Hebbar, and P. V. Vinod, "Plant Disease Detection Using Machine Learning," Proceedings of the International Conference on Design Innovations for 3Cs Compute, Communicate, Control (ICDI3C), IEEE, 2018.

[2] Sharada P. Mohanty, David P. Hughes, and Marcel Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," Frontiers in Plant Science, vol. 7, Article 1419, 2016, doi:10.3389/fpls.2016.01419.

[3] Edna Chebet Too, Li Yujian, Sam Njukia, and Liu Yingchun, "A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification," Computers and Electronics in Agriculture, vol. 161, pp. 272–279, 2019.

[4] Xuewei Sun, Guohou Li, Peixin Qu, Xiwang Xie, Xipeng Pan, and Weidong Zhang, "Research on Plant Disease Identification Based on CNN," Cognitive Robotics, vol. 2, pp. 155–163, 2022.

[5] Natasha Nigar, Hafiz Muhammad Faisal, Muhammad Umer, Olukayode Oki, and Jose Manappattukunnel Lukose, "Improving Plant Disease Classification With Deep-Learning-Based Prediction Model Using Explainable Artificial Intelligence," IEEE Access, vol. 12, 2024, doi:10.1109/ACCESS.2024.3428553.

[6] D. Tejaswi, T. Sri Vaishnavi, B. Nandini, P. Nuka Raju, and B. Deepika, "Plant Disease Detection Using Deep Learning," International Journal of Science and Research Archive, vol. 12, no. 1, pp. 2476–2488, 2024.

[7] Wubetu Barud Demilie, "Plant Disease Detection and Classification Techniques: A Comparative Study of the Performances," Journal of Big Data, vol. 11, Article 5, 2024, doi:10.1186/s40537-023-00863-9.

[8] Quan Zou, Kaiyang Qu, Yamei Luo, Dehui Yin, Ying Ju, and Hua Tang, “Predicting Diabetes Mellitus With Machine Learning Techniques,” Frontiers in Genetics, 2018. (Used as a comparative ML reference in agricultural disease prediction studies)

[9] Md. Kamrul Hasan, Md. Ashraful Alam, Dola Das, Eklas Hossain, and Mahmudul Hasan, “Diabetes Prediction Using Ensembling of Different Machine Learning Classifiers,” IEEE Access, 2020. (Referenced for ensemble learning methodology)

[10] Jyoti Soni, Ujma Ansari, Dipesh Sharma, and Sunita Soni, “Predictive Data Mining for Medical Diagnosis,” International Journal of Computer Applications, vol. 17, no. 8, 2011